

NAME : P.Ajay
HALL NO. : 2403A510B4
BATCH : 04

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week1 - Thursday	Time(s)	
Duration	2 Hours	Applicable to Batches	24CSBTB01 To 24CSBTB39
AssignmentNumber: 2.4(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time	

		to complete
1	Lab 2: Exploring Additional AI Coding Tools – Gemini (Colab) and Cursor AI Lab Objectives: • To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab. • To understand and use Cursor AI for code generation, explanation, and refactoring. • To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI. • To perform code optimization and documentation using AI tools. Lab Outcomes (LOs): After completing this lab, students will be able to: • Generate Python code using Google Gemini in Google Colab. • Analyze the effectiveness of code explanations and suggestions by Gemini. • Set up and use Cursor AI for AI-powered coding assistance. • Evaluate and refactor code using Cursor AI features. • Compare AI tool behavior and code quality across different platforms. Task Description #1 • Open Google Colab and use Google Gemini to generate Python code that performs sorting of a list using both the bubble sort algorithm and Python's built-in sort() function. Compare the two implementations. Expected Output #1 • Two sorting implementations: Bubble sort (manual logic) and Built-in sort() • PROMPT 1: • generate Python code that performs sorting of a list using both the bubble sort algorithm and Python's built-in sort() function. Compare the two implementations. •	Week1 - Thursday

```

# Simple sorting using Python's built-in function
def python_sorted(arr):
    return sorted(arr)

my_list = [5, 2, 8, 1, 9, 4]
sorted_list = python_sorted(my_list)
print("Original list:", my_list)
print("Sorted list (using sorted()):", sorted_list)

# Bubble Sort implementation
def bubble_sort(arr):
    n = len(arr)
    # Create a copy to avoid modifying the original list
    arr_copy = arr[:]
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr_copy[j] > arr_copy[j + 1]:
                arr_copy[j], arr_copy[j + 1] = arr_copy[j + 1], arr_copy[j]
    return arr_copy

my_list_for_bubble_sort = [64, 34, 25, 12, 22, 11, 90]
bubble_sorted_list = bubble_sort(my_list_for_bubble_sort)
print("\nOriginal list for Bubble Sort:", my_list_for_bubble_sort)
print("Sorted list (using Bubble Sort):", bubble_sorted_list)

```

```

Original list: [5, 28, 78, 1, 9, 4]
Sorted list (using sorted()): [1, 4, 5, 9, 28, 78]

Original list for Bubble Sort: [64, 34, 25, 12, 22, 11, 90]
Sorted list (using Bubble Sort): [11, 12, 22, 25, 34, 64, 90]

```

Task Description #2

- In Colab, use Google Gemini to generate a Python function that takes a string and returns: The number of vowels, The number of consonants, The number of digits in the string

Expected Output #2-

- Complete function that Iterates through characters of a string and Counts vowels, consonants, and digits

PROMPT 2 :

- Take a string which contains of digits and alphabets. Then write a python code to count the numbers of digits and constants and vowels can you take string in dyanamic method

```

def count_chars(input_string):
    digits = 0
    vowels = 0
    consonants = 0
    input_string = input_string.lower() # Convert to lowercase for easier vowel checking

    for char in input_string:
        if char.isdigit():
            digits += 1
        elif char.isalpha():
            if char in 'aeiou':
                vowels += 1
            else:
                consonants += 1
    return digits, vowels, consonants

# Example usage:
my_string = input("Please enter a string:") # Get input dynamically from the user
num_digits, num_vowels, num_consonants = count_chars(my_string)

print(f"The string '{my_string}' contains:")
print(f"Digits: {num_digits}")
print(f"Vowels: {num_vowels}")
print(f"Consonants: {num_consonants}")

```

```

Please enter a string:AHUEYDWGU1266213agdfstdfgscb
The string 'AHUEYDWGU1266213agdfstdfgscb' contains:
Digits: 7
Vowels: 5
Consonants: 16

```

Task Description #3

- Install and set up Cursor AI. Use it to generate a Python program that performs file handling:
Create a text file
Write sample text
Read and display the content

Expected Output #3

- Functional code that creates a .txt file, writes content to it, and reads it back.
- Screenshot of Cursor AI interface showing: Prompt used, Generated code, Output of file operations

PROMPT 3 :

- **Write a python programming the create a file and a text in the file and display the content of the file by using file handling**

```
[3] # Define the filename
    filename = "my_example_file.txt"

    # Text to be written to the file
    file_content = "This is some sample text that will be written to the file.\n"
    file_content += "This is the second line."

    # Create and write to the file
    try:
        with open(filename, "w") as file:
            file.write(file_content)
            print(f"Successfully created and wrote to '{filename}'")

    except IOError as e:
        print(f"Error writing to file: {e}")

    # Read and display the content of the file
    try:
        with open(filename, "r") as file:
            read_content = file.read()
            print(f"\nContent of '{filename}':")
            print(read_content)

    except IOError as e:
        print(f"Error reading file: {e}")
```

```
➤ Successfully created and wrote to 'my_example_file.txt'

Content of 'my_example_file.txt':
This is some sample text that will be written to the file.
This is the second line.
```

Task Description #4

- Ask Google Gemini to generate a Python program that implements a simple calculator using functions (add, subtract, multiply, divide). Then, ask Gemini to explain how the code works.

Expected Output #4

- Complete calculator code with user input and operation selection.
- Line-by-line explanation or markdown-style explanation provided by Gemini.
- Screenshot of both the code and explanation in Colab.

Prompt 4:

- Write a python programm that implementations a simple

calculator perform the arithmetic operation. Modify the code and add the comments and explain iterations and explain the code line by line

```
def add(x, y):
    """Adds two numbers."""
    return x + y

def subtract(x, y):
    """Subtracts two numbers."""
    return x - y

def multiply(x, y):
    """Multiplies two numbers."""
    return x * y

def divide(x, y):
    """Divides two numbers."""
    if y == 0:
        return "Error: Cannot divide by zero"
    return x / y

# Get input from the user
try:
    num1 = float(input("Enter first number: ")) # Prompt user for the first number and convert to float
    num2 = float(input("Enter second number: ")) # Prompt user for the second number and convert to float

    print("Select operation:")
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")

    choice = input("Enter choice(1/2/3/4): ") # Prompt user for their choice of operation

    if choice == '1': # Check if the choice is addition
        print(f"{num1} + {num2} = {add(num1, num2)}") # Perform addition and print the result
    elif choice == '2': # Check if the choice is subtraction
        print(f"{num1} - {num2} = {subtract(num1, num2)}") # Perform subtraction and print the result
    elif choice == '3': # Check if the choice is multiplication
        print(f"{num1} * {num2} = {multiply(num1, num2)}") # Perform multiplication and print the result
    elif choice == '4': # Check if the choice is division
        print(f"{num1} / {num2} = {divide(num1, num2)}") # Perform division and print the result
    else:
        print("Invalid input") # Print error for invalid input

except ValueError:
    print("Invalid input. Please enter numbers.") # Print error if input is not a valid number
```

```
Enter first number: 78
Enter second number: 9
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice(1/2/3/4): 2
78.0 - 9.0 = 69.0
```

Explanation :

This Python program is a simple calculator that can add, subtract, multiply, or divide two numbers.

It defines four functions (`add`, `subtract`, `multiply`, `divide`) to perform these operations.

The user enters two numbers and chooses an operation from a menu.

It handles invalid choices and non-numeric inputs using `if-elif-else` and `try-except`.

Task Description #5

- Use Cursor AI to create a Python program that checks if a given year is a leap year or not. Try different prompt styles and see how Cursor modifies its code suggestions.

Expected Output #5

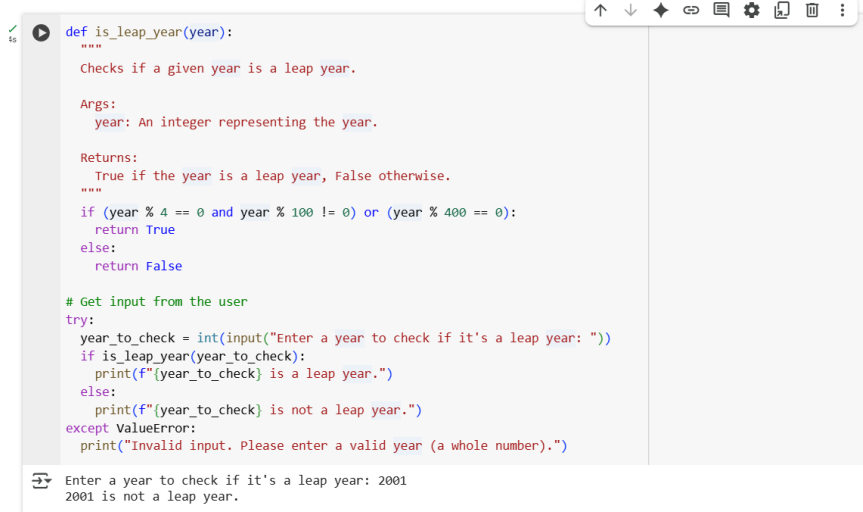
- A functional program to check leap year with sample input/output

- At least two versions of the code (from different prompts)
- A short comparison of which version is better and why

Prompt 5:

- **Write a Python program that checks if a given year is a leap year. Include a function with a docstring, take user input, handle invalid inputs with exceptions, and display whether the year is a leap year or not.**

This code checks if a given year is a leap year.



```
def is_leap_year(year):
    """
    Checks if a given year is a leap year.

    Args:
        year: An integer representing the year.

    Returns:
        True if the year is a leap year, False otherwise.
    """
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False

# Get input from the user
try:
    year_to_check = int(input("Enter a year to check if it's a leap year: "))
    if is_leap_year(year_to_check):
        print(f"{year_to_check} is a leap year.")
    else:
        print(f"{year_to_check} is not a leap year.")
except ValueError:
    print("Invalid input. Please enter a valid year (a whole number).")
```

Enter a year to check if it's a leap year: 2001
2001 is not a leap year.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Two sorting implementations: Bubble sort (manual logic) and Built-in sort() (Task#1)	0.5
Counts vowels, consonants, and digits(Task#2)	0.5
Functional code that creates a .txt file, writes content to it, and reads it back- Use cursor (Task#3)	0.5
Complete calculator code with user input and operation selection. (Task#4)	0.5
A functional program to check leap year with sample input/output-use Cursor (Task#5)	0.5
Total	2.5 Marks